

Git commands used in the course Distributed Version Control with Git (Last update: 01/26/2023)

id	Command	Meaning	Lecture(s)	Frequency
01	git --version	Shows the version of git installed in you local machine.	03	1
02	pwd	"Print Working Directory". Unix command that shows the current directory.	03, 04	2
03	clear	Unix command to clear the console.	03	1
04	git config - -global user.name "Rodrigo Martins Pagliares"	Configures the user that appears when making commits. This is the user shown in all repositories in the local machine for the same logged user. Use git config user.name "Your Name Here" git config user.email your@email.com (without global flag) to configure specific users.	03	1
05	git config - -global user.email "desconhecido@gmail.com"	Configures the user email for that appears when making commits for all repositories in the local machine.	03	1
06	git config - -global core.editor "Notepad.exe"	Configures the default editor on Windows	03	1
07	git config - -global color.ui true	Configure the use of colors in the output of git commands	03	1
08	git config --list	Lists all configurations for this git installation	03	1
09	git config --global	Lists all configurations for this git installation	03	1
10	git config user.email	Shows the configured value for the property <code>user.email</code>	03	1
11	git config user.name	Shows the configured value for the property <code>user.name</code>	03	1
12	ls -la	Unix command that list the contents of a directory including hidden files	03,04, 05, 06	4
13	cat .gitconfig	Shows the content of the hidden file (.gitconfig) with the results of user configurations for the commands git config - -global (commands 5-7)	03	1
14	git --help	Shows the help for git commands	03	1

15	git – help merge	Shows the help for the merge command (in Windows, opens the help in a web browser. In GNU/Linux, shows the help in the terminal)	03	1
16	git init	Initializes a git repository in a directory creating a .git folder in the directory.	04	1
17	notepad .gitignore	On Windows, create the .gitignore file. The entries in this file are not tracked by git (.exe, .class, bin/, etc). If you are using GNU/Linux or Mac OS you can create this file with your editor of preference or by the command <code>touch</code> .	04	1
18	cat .gitignore	The cat command exhibits the contents of a file.	04	1
19	git add .	Adds all changed files in the working directory to the Stage Area	04, 08, 09, 10, 13, 15, 16, 18, 20, 23	11
20	git commit -m "commit message"	Commits to the repository the files in the Stage area. We suggest message with no more than 50 characters.	04, 06, 07, 08, 09, 11, 14, 15, 16, 18, 19, 20, 23	17
21	git log	Shows the log of commits of the git repository	04, 05, 06, 08, 09, 11, 12, 13, 14, 22	22
22	git log --author="Rodrigo"	Shows the log of commits of the git repository made by the user Rodrigo	04	1
23	git log - -grep="versão"	Shows the log of commits of the git repository with the String "versão" in the message of the commit. Grep means Global Regular Expression.	04	1
24	git status	Show the current status of the current branch in the working directory and Stage area	04, 06, 07, 08, 09, 10,	104

			12, 13, 14, 15, 16, 17, 18, 19, 20, 23	
25	cat .git/refs/heads/master	Shows the SHA-1 of the HEAD pointer in the branch master	05	1
26	git add fileName	Adds the file fileName in the working directory to the Stage Area	06, 07, 09, 11, 12, 19	10
27	git diff	Shows the difference between the file(s) changed in the working directory against the file(s) in the repository	07	4
28	git diff fileName	Shows the difference between the fileName changed in the working directory against fileName in the repository .	07, 12	2
29	git diff - -staged	Shows the difference between the file(s) in the Stage Area changed against the file(s) in the repository .	07, 12, 13	4
30	touch nomeArquivo	Unix command to create a new empty file	08, 09	4
31	rm filename	command to remove a file (In Mac/OS the file is gone forever without passing through the trashcan. If you want to remove a file from the terminal and taking it to the trash can, issue <code>mv fileName ~/.Trash/</code>). Issuing a <code>git status</code> just after this command indicates in red this file is not presented in the working directory anymore. We need to place it in the stage area with the command <code>git rm filename</code> and from there commit the deletion to the repository	08	1
32	git rm fileName	Git command to remove a file (the file is inclusive removed from the trashcan). It's better to use <code>git rm fileName</code> instead of <code>rm fileName</code> since it already places the removed file in the stage area, ready to be committed (you can confirm this with the <code>git status</code> command). Being in the stage area, the deletion is ready to be committed.	08, 09	3
33	mv file1 file2	Unix command to rename file1 to file2. If you rename a file that is already in the Stage Area, the command <code>git status</code> will indicate	08, 09	2

		that the file was deleted in the working directory and that a new file (with the new name) is presented in the working directory. Beside that, notice the older file name is still presented in the stage area. You need to issue <code>git rm oldname</code> , in order to eliminate references to it from the working directory and stage area when issuing <code>git status</code> .		
34	<code>git commit - -amend</code>	Opens a file that allows to update the commit message	08, 09	1
35	<code>git mv file1 file2</code>	Rename the file. Same effect as <code>mv file1 file2</code> (command number 33). Suggested way, since after it, the renamed file is placed in the stage area automatically, even if the file is already in the stage area (contras with the example dissed in Line 33 of this table).	08, 09, 11	3
36	<code>mkdir directoryName</code>	Unix command to create a directory	08	1
37	<code>git mv file1 directory/file1Renamed</code>	Moves <code>file1</code> to the directory and at the same rename it	08	1
38	<code>git checkout -- fileName</code>	-- means current branch. Overrides the files of the working directory with the contents of the repository .	10, 12	5
39	<code>git reset HEAD fileName</code>	Removes <code>fileName</code> from stage area . The message that appears when running <code>git status</code> .	10, 12	3
40	<code>git commit --amend -m "message"</code>	Fix the message of the last commit replacing by the one specified here. If you issue this command with some change sets in the stage area (for instance, new files or updates to a file presented in the last commit – HEAD), the effect is to update not only the commit message of the last commit, but also to include the new change set to the last commit in the repository (changing also the SHA-1)	11	1
41	<code>touch file1.txt file2.txt file3.txt</code>	Creates three files at the same time	12	1
42	<code>git clean -n</code>	Indicates which files in the working directory will be removed in case of using <code>git clean -f</code> (this command does not affect files in the stage area – use <code>git reset HEAD filename</code> to unstage it if you want to return it to the working directoy in oder to use <code>git clean</code>). <code>git clean -n</code> returns null if no difference exists between the repository and working directory	12, 17	5

43	git clean -f	Remove the files in the current directory (does not affect files in the stage area). Best practice to precede this command with the git clean -n command	12, 17	3
44	git checkout SHA-1	Checkout the version of the repository flagged with the 40 characters sequence corresponding to the SHA-1. Only first characters is just fine. After issuing this command, the message You are in 'detached HEAD' state appears in the console indicated you are in a previous version of the repository. One easy way to return to the repo state before issuing this command is to type the command <code>git checkout master</code> .	12	1
45	git checkout branchName	Changes to the branch specified in branchName. For instance, git checkout master	12, 15,16, 17, 18,19, 20, 24	25
46	git checkout SHA-1 -- fileName	Checkout fileName from commit with SHA-1 specified, current branch you are in. Since you are back to the past, certify you are positioned in a directory that existed at the time in the past to avoid the error: <code>pathspecc 'filename' did not match any file(s) known to git</code>	12	1
47	git checkout fileName	Checkout the file in the last commit , bringing it to the working directory .	12	1
48	git reset HEAD	Removes the files from stage area . The message that appears when running git status.	13	2
49	cat .git/HEAD	Indicates which branch HEAD is pointing to	13, 15	2
50	cat .git/refs/heads/branchName	Shows the SHA-1 of the commit pointed by HEAD (in the specified branch)	13, 15	4
51	git reset --soft SHA-1	Reverts the repository to the commit with SHA-1. The files created after the commit you revert still appears in the working directory and stage area , since you are using the --soft option. If after resetting to a previous commit, some new commit is made, then it is impossible to recover the commits that existed after the comit	13	1

		target of the reset . Before using git reset, it's a good practice to take a printscreen with the results of the command git log to backup the SHA-1 codes in case you want to revert some modifications. You can still undo the changes by issuing the commando git reset --soft SHA-1 (with the number of the SHA-1 before the first reset taken from the backup printscreen)		
52	git reset --mixed SHA-1	Default option for git reset. Before using git reset, it's a good practice to take a printscreen with the results of the command git log to backup the SHA-1 codes in case you want to revert some modifications. With the mixed option, the state area becomes synchronized with the repository (contrast with the stage area when using the --soft option). You can still undo the changes by issuing the commando git reset --mixed SHA-1 (with the number of the SHA-1 before the first reset taken from the backup printscreen)	13	1
53	git reset --hard SHA-1	Before using git reset, it's a good practice to take a printscreen with the results of the command git log to backup the SHA-1 codes in case you want to revert some modifications. With the hard option, the working directory, stage area and repository becomea all synchronized (contrast with the stage area when using the --mixed option). You can still undo the changes by issuing the commando git reset --hard SHA-1 (with the number of the SHA-1 before the first reset taken from the backup printscreen)	14	3
54	git commit -am "commit message"	Adds to the stage area and committing combined in just one command. It only works with changes in files already tracked by git. If not the case, you need to commit the changes in two steps first.	14, 20	1
55	git branch	Shows the branches in the repository	15, 16, 17, 18, 19, 20, 22, 23, 24	37

56	git log --oneline	Shows the log of the repository (one line per commit)	15, 16, 18, 19, 23	15
57	git branch branchName	Creates a new branch named <code>branchName</code>	15, 16, 18, 23	4
58	ls -la .git/refs/heads	Shows all the branches of the repository (as files in the filesystem)	15	1
59	git checkout branchName	Goes to the specified branch. It's important to have a clean working directory (that has existent files/directories that have been not changed after getting into the branch) before alternating between branches. If you have changed the working directory and don't want to lose the changes when alternating, commit the changes between trying to alternate ou undo the changes before alternate in order to avoid the error: "Your local changes to the following files would be overwritten by checkout". Alternating between branches is possible, "carrying" the changes, that do not conflict, made in the working directory to other branches if the changes are, for instance, the creation of a new file.	15	2
60	git show HEAD	Shows the changes made in the last commit	16	2
61	git checkout -b branchName	Creates and checkout branchName. Notice that the log of commits of the new branch depends on which branch we were located when issuing the command.	16, 18, 20, 24	3
62	git log --graph --oneline --decorate --all	Shows a colored version of the log of commits, indicating the head of each branch	16, 18	2
63	git diff branchName..branchName	Shows the difference between branches. It also works when using remote branches (e.g. origin/master).	17, 19, 23, 24	8
64	git branch --merged	The <code>--merged</code> option filters the output to branches that you have or have not yet merged into the branch you're currently on. Outupt changes depending on which branch you are currently in and the "path" you took to the branch you are currently in (for instance, if you from master alternates to fatorial_recursivo, then to experimentos and then back to fatorial_recursivo, the command	17	3

		will show that you are in <code>fatorial_recursivo</code> (with an <code>*</code>) and list the other two branches as well. <code>Master</code> is the branch with the smallest set of commits. <code>Master</code> is a subset of other branches in number of commits. Notice the <code>*</code> in the output. It indicates the current branch and at the same time the branch with the superset of commit when compared with the log of commits of the other branches.		
65	<code>git branch -move branchOldName branchNewName</code>	Renames branches from old to new.	17, 18	2
66	<code>git branch -d branchName</code>	Removes a branch if you are not in it (outside of it). It is not possible to remove a branch with this command if the branch to be removed that has new commits	18, 24	2
67	<code>git branch -delete branchName</code>	Removes a branch (same as with <code>-d</code> option). It is not possible to remove a branch with this command if the branch to be removed that has new commits	18	1
68	<code>git branch -D branchname</code>	Forces the removal of a branch eve if it has new commits. Contrast with the previous two commands.	18, 19	2
69	<code>git log branchName --oneline -3</code>	Shows the last 3 commits of <code>branchName</code> . It works even if the branch name is remote (e.g <code>origin/master</code>)	19, 20, 23, 24	20
70	<code>git merge branchName</code>	Merges <code>branchName</code> into the branch we are currently in. If only the source branch you are trying to merge into a target branch has new commits, a fast-forward merge is done. If both branches have new commits that do not conflict with each other, a recursive merge is done. In a recursive merge, a commit indicating the merge was done is also addet to the list of commits (<code>git log --oneline</code> shows this "merge" commit). If the two branches you are trying to merge has commits the conflict with each other (same line change, for example), we have a conflict that needs to be solved before the merge (manually or with third-party tools) or alternatively, you can abort the commit. If this happens, you are automatically moved to a temporary branch (<code>master/MERGING</code> , for example). This can be	19, 20	4

		seen with the git status command that indicates we have unmerged paths and need to fix the conflicts. If you open the file(s) with conflict you will notice that git includes some comments on the lines with the conflict(s) in order you decide which version you want to maintain (by manually removing the lines you don't want to keep). After solving the conflict, you need to add to the stage are and make the commit as usual.		
71	git merge --abort	When a merge generates a conflict, we can cancel the merge with this command in order to avoid being the temporary branch o which you manually solves the conflict and return back to the branch you were before issuing the merge command.	20	1
72	git log --oneline -3	Shows the last 3 commits of the current branch	20, 24	4
73	git remote	Show which remote server you already configured. Since we started locally in this course, this returns nothing the first time the command is issued.	21	4
74	git remote add origin urlRepoGitHub	Adds to the local git repository a reference to a remote git repository. After adding a remote repository, it will be shown as the result of issuing the git remote command (command on Line 73 of this table).	21	2
75	git remote -v	Shows addition information about the remote repository configured locally with previous command (command on Line 74 of this table).	21	1
76	cat .git/config	Config file that describes, among other things, the references to the remote repositories/branches	21, 22, 23, 24	6
77	git remote rm origin	git remote rm does not delete the remote repository from the server. It simply removes the remote and its references from your local repository.	21	1
78	git push -u origin master	Sends the contents of a branch (master in this example) to github (origin). The -u option has the goal to keep both branches, master local and remote, synchronized. We can verify this in the local file	22	1

		.git/config that includes an entry indicating a branch is associated with the remote branch at GitHub. <pre>[branch="master"] remote = origin merge= refs/heads/master</pre> One benefit of synchronizing the branches is that we can type fewer options when pushing changes to GitHub (See push command on line 85 for an example). Notice that Git never tries to make a merge in the server after a push. If the server has changes since last git pull or fetch, we first need to pull or fetch the changes (See Lines 86 and 88), make the modifications locally and then send back to GitHub		
79	cat .git/refs/remotes/origin/master	Shows the SHA-1 of the origin/master branch. Notice the remote branches are preceded with the origin/ substring (e.g. origin/master)	22	1
80	git branch -r	Shows the remote branches (e.g. origin/master)	22, 24, 25	6
81	git branch -a	Shows both local and remote branches (e.g. master, origin/master)	22, 24, 25	3
82	git clone urlRepoGitHub	Makes a copy of a remote repository into the local machine. If you want you can specify the name of the directory in the local machine to where the repository will be cloned (e.g git clone urlRepoGitHub directoryxyz) clones the project to directoryxyz to be created)	22	1
83	git push origin branchname	Pushes branch name to remote origin without synchronizing them. The branch name is not shown when we use cat .git/config (in case of using git push without the -u option in the first opportunity.	23	1
84	git log --oneline origin/branchName	Shows the log of commits in the remote branch. For example, git log --oneline origin/master	23	1

85	git push	It works without argument in the case of a synchronized branch (local and remote). The synchronized branch is created at the first push with the -u option	23	1
86	git fetch origin	Fetches the latest updates in the repository without merging locally	24	1
87	git merge branchname branchname	Merges the two braches. For example git merge origin/master master	24	1
88	git pull remoteBranch localBranch	One command that combines git fetch and git merge. For example, git pull origin/master master	24	1
89	git branch branchName origin/branchName	Sets up branchName to track remote branch origin/branchName. For example, git branch sem_sincronismo origin/sem_sincronismo (suppose that before issuing this command, git branch does not reveal any branch name sem_sincronismo locally, but this branch exists in origin (origin/sem_sincronismo). After that, the git branch locally reveals the local branch sem_sincronismo and an entry indicating the branches local and remote are synchronized appears in the local file .git/config	24	1
90	Git checkout -b localBranch remoteBranch	Sets up branchName to track remote branch origin/branchName and at the same time alternates to localBranch.	24	1
91	git push origin :branchname	Removes the remote branch.	25	1

PALESTRA

id	Command	Meaning	Frequency
01	git help git	Shows the git manual	1
02	git help command	Shows the help for the command	2
03	git config - -global user.name "Rodrigo Martins Pagliares"	Configures the user that appears when making commits	2
04	git config - -global user.email "desconhecido@gmail.com"	Configures the user email that appears when making commits	2
05	git config --list	Lists all configurationf used by git	2
06	git init	Initializes a git repository in a directory creating a .git folder in the direcotory.	1
07	git status	Show the current status of the current branch in the working directory and Stage area	14
08	git add .	Adds all changed files in the working directory to the Stage Area	3
09	git commit -m "commit message"	Commits to the repository the files in the Stage area	1
10	git log	Shows the log of commits of the git repository	9
11	ssh-keygen -C "email" -t rsa	Generates public/private rsa key par. We need to copy its contents to the GitHub (SSH keys menu)	2
12	git remote	Shows a list of existing remote repositories	2
13	git remote add origin urlRepoGitHub	Adds to the local git repository a reference to a remote git repository. After adding a remote repository, it will be shown as the result of issuing the git remote command (command on Line 73 of this table).	1
14	git push -u origin master	Sends the contents of a branch (master in this example) to github (origin). The -u option has the goal to keep both branches, master local and remote, synchhronized. We can verify this in the local file .git/config that includes an	2

		entry indicating a branch is associated with the remote branch at GitHub. <pre>[branch="master"] remote = origin merge= refs/heads/master</pre> One benefit of synchronizing the branches is that we can type fewer options when pushing changes to GitHub (See push command on line 85 for an example). Notice that Git never tries to make a merge in the server after a push. If the server has changes since last git pull or fetch, we first need to pull or fetch the changes (See Lines 86 and 88), make the modifications locally and then send back to GitHub	
15	git clone git@github.com:pagliaresbcc/PalestraGit.git	Makes a copy of a remote repository into the local machine. If you want you can specify the name of the directory in the local machine to where the repository will be cloned (e.g <code>git clone urlRepoGitHub directoryxyz</code>) clones the project to <code>directoryxyz</code> to be created)	1
16	git branch	Shows the branches in the repository	6
17	git checkout -b ramificacao_testes	Creates and checkout branchName. Notice that the log of commits of the new branch depends on which branch we were located when issuing the command.	2
18	git commit -a -m "commit message"	Add to the stage area and commit to the repository with one single command instead of two (it used to work at the time of the recording. Need to verify if this command works in newer versions of Git)	3
19	git add fileName	Adds the file fileName in the working directory to the Stage Area	1

20	git checkout branchName	Changes to the branch specified in branchName. For instance, git checkout master	2
21	git rebase ramificacao_testes	Command issued on branch master. Changes to the branch specified in branchName. For instance, git checkout master	1
22	git push origin master	Pushes master content to the master branch in the remote repository	2
23	git pull origin master	pull changes from the locally stored branch origin/master and merge that to the local checked-out branch	1
24	git merge branchName	Merges the content of branchName into the branch you are currently in	1

Additional commands (not discussed on the screencasts)

ID	Command	Meaning
01	<code>git status -uall</code>	Useful when git status is identifying a new untracked folder, but it is not showing its contents.
02	<code>git tag</code>	Listing the existing tags in the repository in alphabetical order
03	<code>git tag -l "v1.8.5*"</code>	Listing the existing tags in the repository searching for tags that match a particular pattern. <pre>pagliares@Rodrigos-MacBook-Pro concrete-factory-technique % git log --oneline 9d8a241 (HEAD -> tag, tag: v1.0, origin/main, origin/HEAD, main) Create initial version of the example bee6d81 Update README c99850b Add InventotySystem.jar to the lib folder</pre> Despite that, we pull request to GitHub
04	<code>git tag -a v1.0 -m "tag description"</code>	Create a tag Example: <code>git tag -a v1.0 -m "Initial version of the code evolved during the class".</code> (Does it change the commit history? Git status after the command does not identify the tag command) It looks it does not create another commit, only update the metadata of the last commit (pointed by HEAD) to include tag data. Example:
05	<code>git show <tag name></code>	You can see the tag data along with the commit that was tagged by using the git show command: <pre>pagliares@Rodrigos-MacBook-Pro concrete-factory-technique % git show v1.0 tag v1.0 Tagger: Rodrigo Martins Pagliares <pagliares@bcc.unifal-mg.edu.br> Date: Fri Oct 28 13:16:21 2022 -0300 Initial version of the code evolved during the class commit 9d8a2415b5d194b0095a404df44685ca7b5329d0 (HEAD -> tag, tag: v1.0, origin/main, origin/HEAD, main) Author: Rodrigo Martins Pagliares <pagliares@bcc.unifal-mg.edu.br> Date: Fri Oct 28 12:29:08 2022 -0300 Create initial version of the example</pre>

06	git tag v1.4-lw	To create a lightweight tag, don't supply any of the -a, -s, or -m options, just provide a tag name. if you run git show on the tag, you don't see the extra tag information. The command just shows the commit: <pre>\$ git show v1.4-lw commit ca82a6dff817ec66f44342007202690a93763949 Author: Scott Chacon <schacon@gee-mail.com> Date: Mon Mar 17 21:52:11 2008 -0700 Change version number</pre>
07	git tag -a <tagname> <sha-1>	Tag a specific commit. To see the effects : git tag, git show v1.2 (example: git tag -a v1.2 sha1)
08	git push origin <tagname>	Push tags to a shared server after you have created them. This process is just like sharing remote branches — you can run git push origin <tagname>. <pre>pagliares@Rodrigos-MacBook-Pro concrete-factory-technique % git push origin v1.0 Counting objects: 1, done. Writing objects: 100% (1/1), 207 bytes 207.00 KiB/s, done. Total 1 (delta 0), reused 0 (delta 0) To https://github.com/pagliares/concrete-factory-technique.git * [new tag] v1.0 -> v1.0</pre>
09	git push origin --tags	Push a lot of tags that you want to push up at once <pre>\$ git push origin --tags Counting objects: 1, done. Writing objects: 100% (1/1), 160 bytes 0 bytes/s, done. Total 1 (delta 0), reused 0 (delta 0) To git@github.com:schacon/simplegit.git * [new tag] v1.4 -> v1.4 * [new tag] v1.4-lw -> v1.4-lw</pre>

10	<code>git tag -d <tagname></code>	To delete a tag on your local repository \$ <code>git tag -d v1.4-lw</code> Deleted tag 'v1.4-lw' (was e7d5add) Note that this does not remove the tag from any remote servers.
11	<code>git push <remote> :refs/tags/<tagname>:</code>	There are two common variations for deleting a tag from a remote server. \$ <code>git push origin :refs/tags/v1.4-lw</code> To /git@github.com:schacon/simplegit.git - [deleted] v1.4-lw
12	\$ <code>git push origin --delete <tagname></code>	The second (and more intuitive) way to delete a remote tag is with:

Git supports two types of tags: lightweight and annotated. A lightweight tag is very much like a branch that doesn't change — it's just a pointer to a specific commit. Annotated tags, however, are stored as full objects in the Git database. They're checksummed; contain the tagger name, email, and date; have a tagging message; and can be signed and verified with GNU Privacy Guard (GPG). It's generally recommended that you create annotated tags so you can have all this information; but if you want a temporary tag or for some reason don't want to keep the other information, lightweight tags are available too.

By default, the `git push` command doesn't transfer tags to remote servers. You will have to explicitly push tags to a shared server after you have created them. This process is just like sharing remote branches — you can run `git push origin <tagname>`.

If you have a lot of tags that you want to push up at once, you can also use the `--tags` option to the `git push` command. This will transfer all of your tags to the remote server that are not already there. Now, when someone else clones or pulls from your repository, they will get all your tags as well. On GitHub the tags are shown in the release panel at the right of the figure.